Python Quick Revision Notes & Exam Cheat Sheet

Synthesized Key Definitions, Common Pitfalls, and High-Frequency Formulas

1. Variable Scoping: The LEGB Rule

When an identifier is referenced in Python, the interpreter searches four scopes hierarchically: 1. **Local (L):** Defined inside current function. 2. **Enclosing (E):** Found in outer nested enclosing functions (closures). 3. **Global (G):** Declared at the top level of the module script. 4. **Built-in (B):** Pre-assigned names in Python (`len`, `range`, `Exception`). To rebind a global variable inside a function, declare `global var_name`. To rebind an enclosing variable in a closure, declare `nonlocal var_name`.

2. Top 5 Fatal Examination Traps to Avoid

Trap 1: Mutable Default Arguments

Default parameter expressions are evaluated once when the function is defined, NOT each time it is called. Writing `def append_to(item, target=[])` causes all calls sharing the default to mutate the same list object across invocations. Solution: Use sentinel `target=None` and initialize `if target is None: target = []` inside the function body.

Trap 2: `is` vs `==`

`==` compares values/content (calls `__eq__`). `is` tests memory address identity (`id(a) == id(b)`). Never use `is` to compare strings or integers beyond small-integer caching (-5 to 256). Always compare singletons (`None`, `True`, `False`) with `is`.

Trap 3: Modifying a Collection While Iterating Over It

Deleting items from a list while looping over it using `for x in my_list:` shifts indices, skipping adjacent items or producing runtime errors. Solution: Iterate over a shallow slice copy: `for x in my_list[:]:` or construct a list comprehension.

Trap 4: Shallow Copy vs Deep Copy

`list.copy()` or `my_list[:]` creates a new outer list, but inner compound objects remain shared references. If a nested sub-list is modified in the copy, the original changes too. Use `copy.deepcopy()` for nested structures.

3. Dunder (Magic) Methods Quick Reference

Method	Triggered By	Primary Purpose
__init__(self, ...)	MyClass(...)	Initializes instance state after allocation
__str__(self)	str(obj), print(obj)	Human-readable end-user string representation
__repr__(self)	repr(obj), interactive REPL	Unambiguous developer representation (eval-able)
__len__(self)	len(obj)	Returns non-negative integer container size
__getitem__(self, k)	obj[k]	Implements indexing and sequence/mapping retrieval
__iter__(self)	iter(obj), for x in obj	Returns an iterator object implementing __next__
__enter__ & __exit__	with obj as x:	Defines context manager setup and deterministic teardown

4. Essential Decorator Boilerplate Template

```
from functools import wraps def time_execution(func): @wraps(func) # Preserves
func.__name__ and docstrings def wrapper(*args, **kwargs): import time start =
time.perf_counter() result = func(*args, **kwargs) elapsed = time.perf_counter() - start
print(f'{func.__name__} executed in {elapsed:.4f}s') return result return wrapper
```